POLITECNICO SETUBAL

Technical Document 2 - Animation with Code

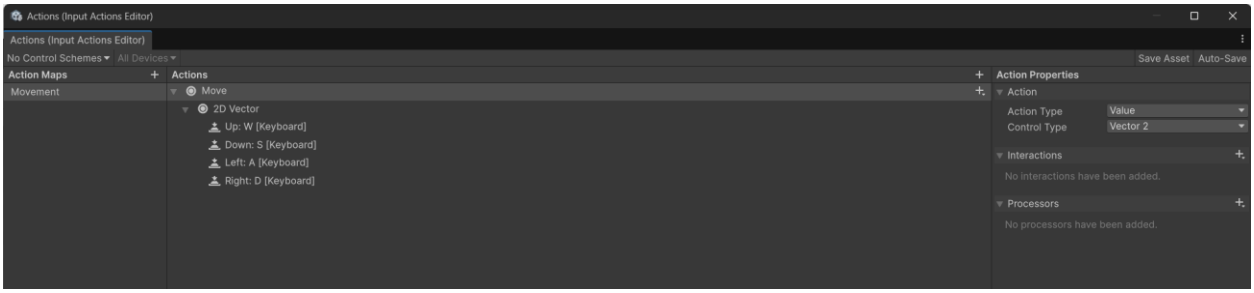
Design e Programação de Jogos (DPJ)

Level 1

We started by creating a Unity project called "DPJ_Lab2", then we created a folder called "Action" and created an ActionMap inside it called "Actions".

In the action map, we created an Action called Move with the action type as Value and the Control type as Vector2.

Finally, we added the binding "Add up/Down/Left/Right Composite",and made it so that we're able to control each movement using the WASD keys.



Level 2

In the second level, we located the sprites for the "green ninja" and placed all the definitions as follows:

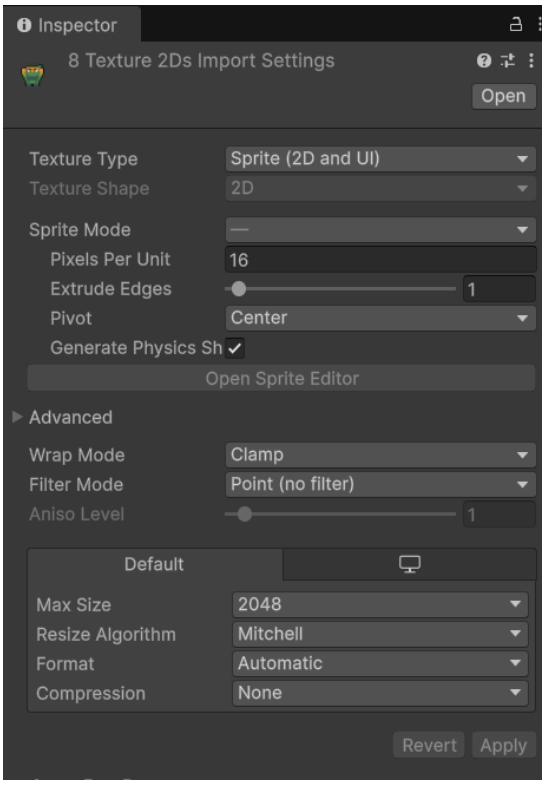
Pixels per Unit: 16;

Filter Mode: Point (no filter)

Compression: None;

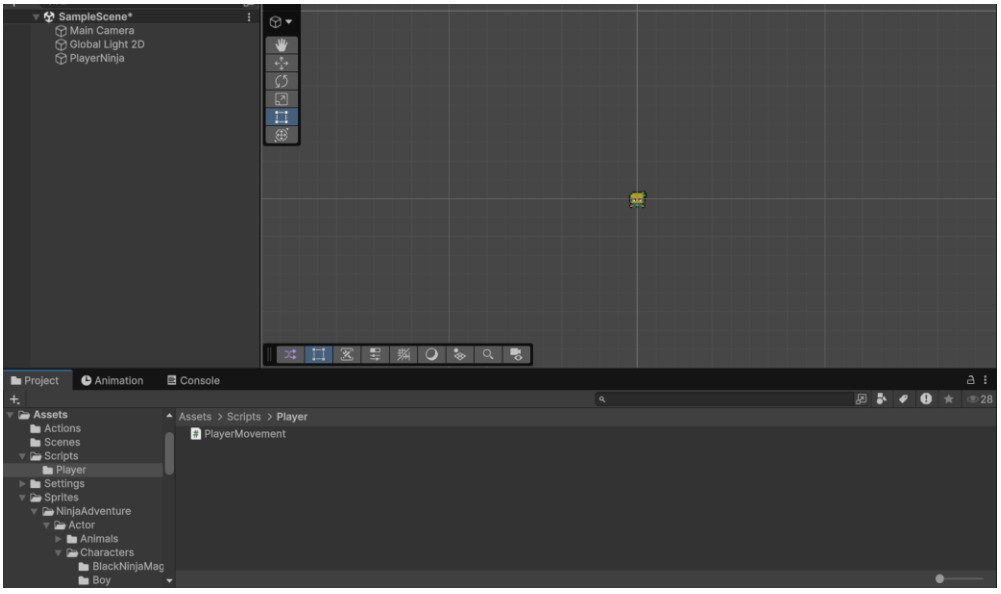
We also selected the spritesheets, walk, jump, idle, and attack and placed them as Sprite Mode: Multiple, leaving the others as "Sprite mode: Single".

Finally, we used the sprite Sheet Editor on the Sprite "walk" to slice each frame of animation.



Level 3

In this level, we dragged the first sprite of the walk spritesheet into the hierarchy of the scene and called the sprite “PlayerNinja”, then inside the SpriteRenderer of the Player, we created a new Sorting Layer called “Player” and added it. Finally, we made a C# file called “PlayerMovement.cs” and added it to the PlayerNinjacomponent.

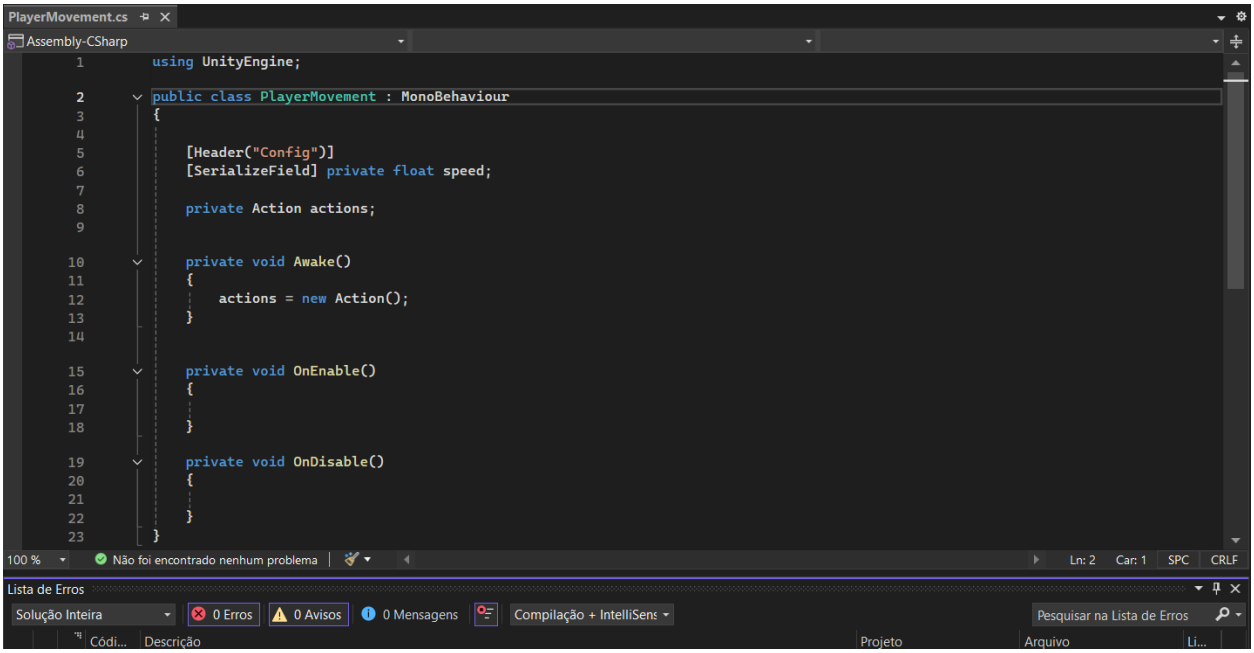


Level 4

In this level we added code inside the C# script “PlayerMovement.cs”. Inside it we have a header called “Config”, it serves as a simple header on Unity.

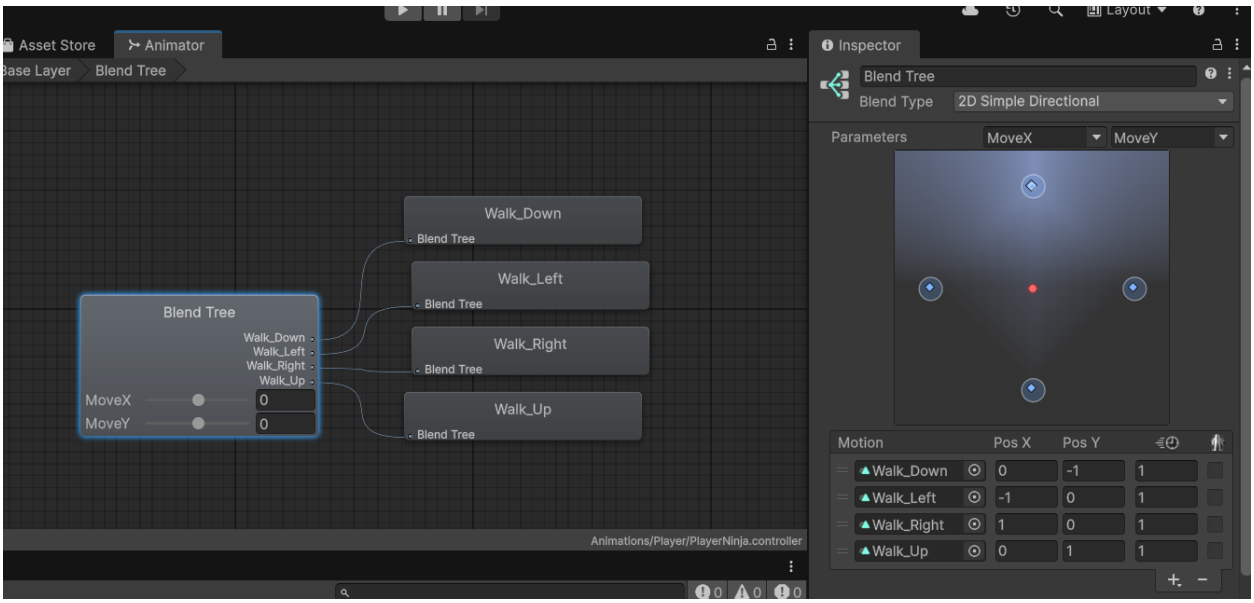
Then we made the variable called “speed” that will regulate the player’s speed. We made a function called Awake that tells the game, that when the game is running, the actions need to be the first thing on the list to activate.

Finally, we have two empty functions called OnEnable and OnDisable that will tell the machine what will happen the input actions are enabled or disabled.



Level 5

We started making the walking animations for the Green Ninja (Player), we improved the code so it would show the animations to the desired direction of the Player walking (Up,Down,Left,Right), in addition we made a blend tree for the animation for them to play when the player faces a direction.



In the image above there's the Blend tree, that's commonly used to make the animations function fluently without error.

It's very important because it eliminates clunky state transitions, ensuring smooth visual updates that instantly matches the player input without needing dozens of complex Animator code conditions.